Managing Traffic with OpenShift Service Mesh

Lesson | Lab Environment

Chapter 1. Managing Traffic with OpenShift Service Mesh

Abstract

Goal	Manage, control, and test application traffic in Red Hat OpenShift Service Mesh by applying routing strategies, resiliency policies, and fault injection techniques to build safer and more reliable distributed systems.
Sections	• Introduction to OpenShift Service Mesh Traffic Management (and Guided Exercise) • Traffic Routing and Splitting with OpenShift Service Mesh (and Guided Exercise) • Fault Injection with OpenShift Service Mesh (and Guided Exercise) • Resilience with OpenShift Service Mesh (and Guided Exercise) • Traffic Mirroring with OpenShift Service Mesh (and Guided Exercise) • Introduction to the Kubernetes Gateway API (and Guided Exercise)
Lab	• Managing Traffic with OpenShift Service Mesh

Introduction to OpenShift Service Mesh Traffic Management

Objectives

- Explain how OpenShift Service Mesh decouples traffic logic from application code and the benefits of this separation.
- Describe how traffic flows through OpenShift Service Mesh components.
- Identify the purpose and use cases for the core OpenShift Service Mesh traffic management resources.

Introduction to OpenShift Service Mesh Traffic Management

Red Hat OpenShift Service Mesh provides traffic management capabilities that control the flow of traffic and API calls between services. You can decouple traffic policies from application code, gaining fine-grained control over request routing in your microservices architecture.

Traditional service-to-service communication in Kubernetes relies on basic load balancing across pod endpoints. Although this works for simple scenarios, modern microservices architectures require more sophisticated traffic control.

OpenShift Service Mesh traffic management provides the following key benefits:

Decouple traffic policies from application code

Configure routing, retries, timeouts, and other traffic behaviors without modifying application code. This separation enables operations teams to manage traffic policies independently from development teams.

Gain fine-grained control over traffic flow
Route traffic based on HTTP headers, URI paths, source labels, or other request attributes. For example, you can send all requests from internal users to a specific service version, or route mobile traffic differently from web traffic.

Enable progressive delivery
Implement deployment strategies such as canary releases and A/B testing by directing a percentage of traffic to new versions. Gradually increase traffic to a new version as you monitor its behavior, reducing the risk of widespread failures.

Improve application resiliency
Configure automatic retries, circuit breakers, and timeouts to protect services from transient failures. These reliability features help applications handle failures in dependent services or network issues.

Mirror traffic for testing and analysis
Copy live traffic to a secondary service without impacting production requests. Traffic mirroring enables testing new service versions with real-world data as the primary service continues handling user requests. This is valuable for validating performance, testing features, or analyzing traffic patterns without risk to users.

How OpenShift Service Mesh Manages Traffic

OpenShift Service Mesh traffic management is built on three core components:

Envoy Sidecar Proxy
All traffic entering and leaving application pods flow through an Envoy proxy deployed as a sidecar container. The Envoy proxy intercepts network communication, enabling OpenShift Service Mesh to:

- Apply traffic routing rules
- Collect telemetry data
- Enforce security policies
- Implement resiliency patterns

Because all traffic passes through Envoy, you can change traffic behavior without modifying application code or redeploying services.

OpenShift Service Mesh Control Plane (Istiod)
The control plane, Istiod, manages and configures all Envoy proxies in the service mesh. When you create traffic management resources such as virtual service or destination rule, Istiod does the following:

1. Validates the configuration
2. Converts it into Envoy-specific configuration
3. Distributes the configuration to the appropriate Envoy proxies

This centralized control plane ensures that all proxies have a consistent view of the traffic management rules.

Data Plane
The data plane consists of all Envoy proxies deployed alongside application workloads. These proxies enforce traffic management rules received from the control plane. The data plane operates independently, so traffic continues flowing even if the control plane is temporarily unavailable.

OpenShift Service Mesh Traffic Management Architecture

The following diagram illustrates traffic flow through OpenShift Service Mesh, from external clients through the ingress gateway, between services within the mesh, and out to external services through egress.

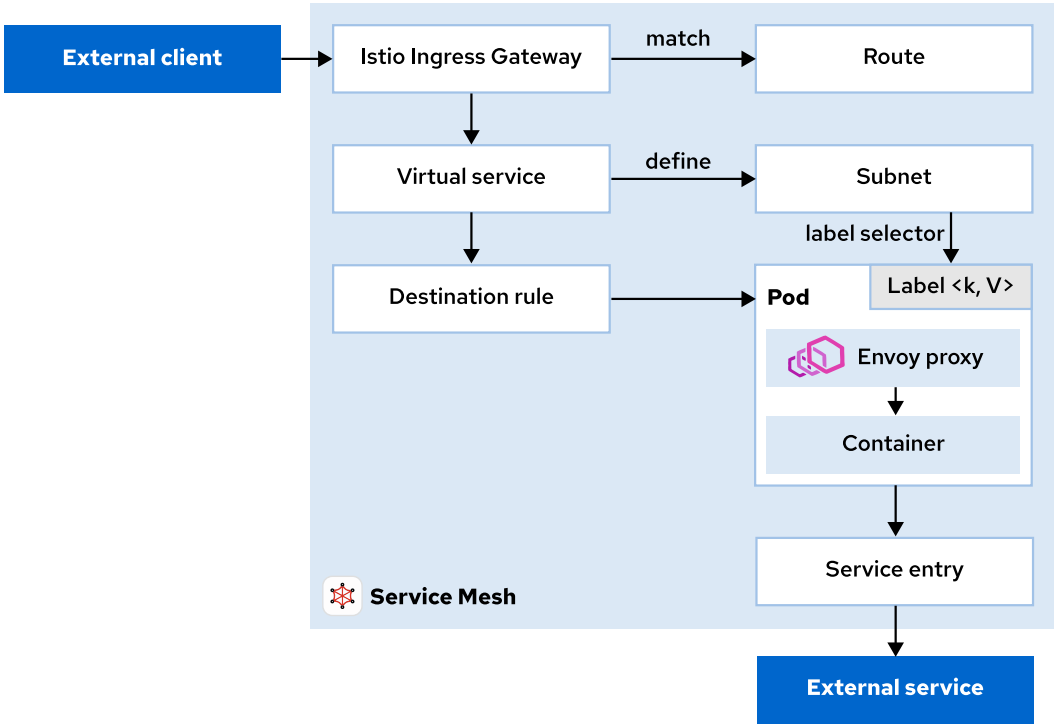

Figure 1.1: OpenShift Service Mesh Traffic Management Architecture

Ingress Traffic Flow
External traffic enters the service mesh through an Istio gateway resource, which configures the ingress gateway pods to accept traffic on specific ports and protocols.

A virtual service then routes incoming traffic to backend services based on request attributes such as HTTP headers or URI paths.

Internal Mesh Traffic Flow
Services within the mesh communicate through their Envoy sidecars. You can use virtual service and destination rule resources to control how traffic flows between services, including:

- Routing traffic to specific service versions

- Splitting traffic across multiple versions
- Implementing retries and timeouts
- Applying load balancing policies

Egress Traffic Flow
By default, OpenShift Service Mesh permits services to access external endpoints outside the mesh. You can configure the mesh to block all egress traffic and selectively permit access to specific external services by using service entry resources.

This approach provides better security and visibility into external dependencies.

Core OpenShift Service Mesh Traffic Management Components

OpenShift Service Mesh provides resources for managing traffic within the service mesh. This section explores the five core resources that control traffic flow.

Istio Gateway

A gateway resource configures a load balancer to receive incoming traffic from outside the mesh or send traffic outside the mesh. It defines ports, protocols, and accepted hosts.

The gateway resource defines the following attributes:

- Port configuration: number, name, protocol
- Accepted hostnames: can use wildcards
- TLS settings: for HTTPS traffic
- Selector: to specify which gateway pods apply this configuration

A gateway by itself does not route traffic to backend services. You must bind a virtual service to the gateway to define routing rules.

OpenShift Service Mesh deploys ingress gateway pods at the mesh edge to handle incoming traffic. These gateway pods typically run in the `istio-ingress` namespace with the `istio: ingressgateway` label. When creating a gateway resource, use the `selector` field to specify which gateway pods apply this configuration.

The following example creates a gateway that accepts HTTP traffic on port 80 for any hostname:

```
apiVersion: networking.istio.io/v1
kind: Gateway
metadata:
  name: my-gateway
spec:
  selector:
    istio: ingressgateway (1)
  servers:
  - port:
      number: 80 (2)
      name: http
      protocol: HTTP
    hosts:
    - "*" (3)
```

1. Specifies which ingress gateway pods should use this configuration. The label `istio: ingressgateway` matches the default OpenShift Service Mesh ingress gateway.
2. Configures the gateway to listen on port 80 for HTTP traffic.
3. Accepts traffic for any hostname. In production, specify actual domain names such as `example.com`.

Virtual Service

A virtual service resource defines routing rules that control request routing to services. It decouples client destinations from backend services that handle requests.

The virtual service resource provides:

- Host configuration to specify which service names this routing applies to
- Gateway binding to connect to ingress/egress gateways
- HTTP routing rules with match conditions based on:
  - URI paths, such as exact, prefix, or regex
  - HTTP headers, including custom headers
  - Query parameters
  - HTTP methods
- Traffic distribution across multiple destinations or service versions
- Traffic manipulation such as URL rewrites and redirects
- Fault injection for testing resiliency

The following virtual service example routes traffic based on the HTTP `end-user` header:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: my-service
spec:
  hosts:
  - "*" (1)
  gateways:
  - my-gateway (2)
  http:
  - match: (3)
    - uri:
        prefix: /api
      headers:
        end-user:
          exact: premium
    route:
    - destination:
        host: my-service
        subset: v2 (4)
  - match: (5)
    - uri:
        prefix: /api
    route:
    - destination:
        host: my-service
        subset: v1
```

1. Accepts traffic from any hostname.
2. Binds this routing configuration to the `my-gateway` Gateway resource.
3. First routing rule: matches requests to `/api` with the header `end-user: premium`.
4. Routes matching requests to the `v2` subset of the service.
5. Second routing rule: matches all other requests to `/api` and routes them to `v1`.

Destination Rule

A destination rule resource defines policies for traffic after routing. It configures load balancing, connection pool settings, and outlier detection for a destination service.

Most importantly, destination rule defines service subsets, which represent different versions of a service based on pod labels.

The destination rule resource provides:

- Service subset definitions based on pod labels
- Load balancing algorithms, such as `ROUND_ROBIN`, `RANDOM`, `LEAST_REQUEST`
- Connection pool settings for HTTP and TCP traffic
- Outlier detection with circuit breaker configuration

The following destination rule example defines two subsets of a service:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: my-service
spec:
  host: my-service (1)
  subsets: (2)
  - name: v1
    labels:
      version: v1
  - name: v2
    labels:
      version: v2
```

(1) Specifies which service this rule applies to.

(2) Defines two subsets (`v1` and `v2`) based on the `version` label on the pods.

After defining subsets, reference them in virtual service resources to route traffic to specific service versions.

Service Entry

A service entry resource adds external services to OpenShift Service Mesh's internal service registry, enabling mesh services to access these external endpoints.

This is important when configuring OpenShift Service Mesh to block egress traffic by default. You can then selectively permit access to specific external services.

The service entry resource provides:

- External host configuration, such as DNS names or IP addresses
- Port and protocol definitions
- Location specification, external to mesh or internal
- DNS resolution mode, such as DNS lookup, static IP, or none
- Endpoint definitions for specific IP addresses

The following service entry example permits mesh services to access an external `out.example.com` service:

```
apiVersion: networking.istio.io/v1
kind: ServiceEntry
metadata:
  name: my-external-se
spec:
  hosts: (1)
    - out.example.com
  ports: (2)
    - number: 80
      name: http-port
      protocol: HTTP
  location: MESH_EXTERNAL (3)
  resolution: DNS (4)
```

1. Specifies the external hostname that mesh services can access.
2. Defines the port and protocol for communicating with the external service.
3. Indicates that this service is external to the mesh.
4. Instructs Envoy to use DNS resolution to find the service's IP address.

Sidecar

A sidecar resource optimizes Envoy proxy sidecars by limiting the services they can reach and ports they accept. This resource is used for performance optimization in large-scale deployments.

By default, OpenShift Service Mesh configures every Envoy proxy to accept traffic on all ports and to reach every service in the mesh. In large applications with hundreds of services, this can lead to high memory usage in each proxy.

The sidecar resource provides:

- Fine-tuning of the ports and protocols an Envoy proxy accepts
- Limiting which services an Envoy proxy can reach
- Namespace-wide or workload-specific configuration
- Reduced memory footprint for proxies in large meshes

The following sidecar example configures all services in a namespace to reach only services in the same namespace and the Istio control plane:

```
apiVersion: networking.istio.io/v1
kind: Sidecar
metadata:
  name: default
  namespace: my-namespace
spec:
  egress: (1)
  - hosts:
    - "./" (2)
    - "istio-system/" (3)
```

1. Configures egress traffic from the sidecar.
2. Permits access to all services in the current namespace. The `./*` represents the current namespace.
3. Permits access to services in the `istio-system` namespace, needed for control plane communication.

> **NOTE**
>
> Use the sidecar resource only to optimize performance in large deployments or implement strict network segmentation.

Comparison of OpenShift Service Mesh Traffic Management Resources

The following table summarizes the purpose and key use cases for each OpenShift Service Mesh traffic management resource:

Resource	Primary Purpose	Key Use Cases
Gateway	Configure ingress/egress gateway load balancers	• Accept external traffic into the mesh • Configure HTTPS/TLS termination • Expose services on specific ports
Virtual service	Define routing rules for traffic	• Route traffic based on request attributes (URI, headers) • Implement canary deployments and A/B testing • URL rewrites and redirects • Traffic splitting across versions
Destination rule	Configure policies for a destination service	• Define service subsets • Configure load balancing algorithms • Set connection pool limits • Implement circuit breakers
Service entry	Add external services to the mesh registry	• Access external APIs from within the mesh • Control egress traffic to specific external endpoints • Integrate non-mesh services, such as VMs and external databases
Sidecar	Optimize sidecar proxy configuration	• Limit which services a proxy can reach • Reduce memory usage in large deployments • Implement network segmentation at the proxy level

Kubernetes Gateway API

In addition to the Istio API, OpenShift Service Mesh 3.1 supports the Kubernetes Gateway API.

The Kubernetes Gateway API is an official Kubernetes project that provides a standard way to manage Layer 4 and Layer 7 routing in Kubernetes. It represents the next generation of Kubernetes ingress, load balancing, and service mesh APIs. Its design is role-oriented, expressive, and portable across different implementations.

Istio supports the Gateway API and intends to make it the default API for traffic management in the future. OpenShift Service Mesh *ambient mode* requires the Kubernetes Gateway API.

IMPORTANT

Note that there are no plans to remove the stable Istio APIs, such as virtual service, destination rule, and others.

NOTE

This course focuses primarily on the Istio API because it provides complete feature coverage and is the most widely documented approach for OpenShift Service Mesh. A dedicated lecture and guided exercise cover the Kubernetes Gateway API in detail. Refer to the the section called " Introduction to the Kubernetes Gateway API " (/rol/app/courses/ad0008l-3.1/pages/ch01s11) for more information.

REFERENCES

For more information about the OpenShift Service Mesh Kubernetes Gateway API support, refer to the *Support for Kubernetes Gateway API* section in the *Red Hat OpenShift Service Mesh version 3.1 new features and enhancements* chapter in the OpenShift Service Mesh 3.1 of *Release Notes* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/release_notes/index#support-for-kubernetes-gateway-api_ossm-release-notes (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/release_notes/index#support-for-kubernetes-gateway-api_ossm-release-notes)

Introducing OpenShift Service Mesh 3.1 (https://www.redhat.com/en/blog/introducing-openshift-service-mesh-31)

Istio.io: Istio Traffic Management Concepts (https://istio.io/v1.26/docs/concepts/traffic-management/)

Istio.io: Istio Traffic Management Best Practices (https://istio.io/v1.26/docs/ops/best-practices/traffic-management/)

Istio.io: Istio Kubernetes Gateway API Task (https://istio.io/v1.26/docs/tasks/traffic-management/ingress/gateway-api/)

Istio.io: Gateway
(https://istio.io/v1.26/docs/reference/config/networking/gateway/)

Istio.io: Virtual Service
(https://istio.io/v1.26/docs/reference/config/networking/virtual-service/)

Istio.io: Destination Rule
(https://istio.io/v1.26/docs/reference/config/networking/destination-rule/)

Istio.io: Service Entry
(https://istio.io/v1.26/docs/reference/config/networking/service-entry/)

Istio.io: Sidecar
(https://istio.io/v1.26/docs/reference/config/networking/sidecar/)

Revision: ad0008l-3.1-ed4-20260310-8ae9760